Deliverable 2: Interim Progress Report

Cybersecurity Analytics Class Project

Deliverable	*Deliverable 2: Interim Progress Report 1*	**Selected Track**	IDS (Intrusion Detection Systems)
Project Title	ScanShield: Detecting Network Port Scanning Using Flow-Level Traffic Analysis	**Team Name**	NetSentries
Team Members	Riya Sanjiv Pradhan; Ojas Prashant Suratkar	**Submission Date**	October 11, 2026
Repository	https://github.com/riyap3111/ScanShield	**Dataset Link**	https://www.kaggle.com/datasets/sateeshkumar6289/cicids-2017-dataset

Executive Status Snapshot

Item	Current status
Overall status	On track. Data audited, cleaned and split; EDA done; baselines trained on all 79 features; known-tester allowlist and slow-scan features implemented and tested. Random Forest and XGBoost next.
Most important accomplishment	Reproducible pipeline (286,467 → 213,602 flows); logistic regression on all features catches 99.8% of scans with 6 false alarms in 18,436 benign validation flows.
Most important risk	Scores may reflect tool-specific artifacts (1,024-byte TCP window in 49.6% of scans, 0.0% of benign); the allowlist and slow-scan controls need data with source IPs and timestamps to be evaluated.
Next milestone	Random Forest and XGBoost trained, tuned and tested for robustness by October 25, 2026; Status Report on November 8, 2026.

1. Project Status and Changes from Proposal

Since the proposal, the project has moved from planning to a working, reproducible pipeline. The team audited the CICIDS2017 PortScan file without modifying it, cleaned it in eight logged steps, completed the EDA, built the feature sets and splits, and trained four baselines. Two risk controls from the proposal, an allowlist for approved internal scanners and slow-scan features, are also now implemented; run_all.sh reproduces every result here in under two minutes (seed 42). Five changes were made:

- **Class balance.** The proposal expected benign traffic to dominate. The raw file is 55.5% PortScan, but after removing duplicates it is 57.5% benign and 42.5% PortScan. The imbalance is mild, so stratified splits and class weights replace resampling.
- **All columns are kept.** The proposal planned to narrow the 78 features. The team instead models on every column so no useful signal is lost; a pruned 43-feature set is kept only for comparison (validation false alarms: 17 pruned vs 6 all).
- **Outside-day test data.** The Monday file (529,918 benign flows) measures false alarms on another day; it is never used for training.
- **Split strategy.** The stratified 70/15/15 split remains primary; a time-ordered split was added because scans arrive in one burst.

- **Risk mitigations moved into code.** Internal scanners and slow scans, listed as risks in the proposal, now have tested solutions (Section 7, Appendix B).

Scope, success criteria (scan recall ≥ 0.90, benign FPR ≤ 5%), roles and schedule are unchanged and feasible; with both criteria already met, work shifts to robustness and deployment controls. No written instructor feedback on the proposal had been returned when this report was prepared.

2. Dataset Description and Data Dictionary

The data is CICIDS2017 from the Canadian Institute for Cybersecurity, University of New Brunswick (Sharafaldin et al., 2018; Canadian Institute for Cybersecurity, n.d.), obtained from a Kaggle mirror of the MachineLearningCVE release (sateeshkumar6289, n.d.). Simulated users generated benign traffic on a lab network from July 3 to 7, 2017, a separate network launched the attacks, and CICFlowMeter converted the captures into bidirectional flows (Lashkari et al., 2017). It is public for research with citation and holds no real user data.

The main file (Friday-WorkingHours-Afternoon-PortScan, the afternoon of July 7, 2017) has 286,467 records; each record is one network flow, the unit of analysis. It has 79 columns: 78 numeric features and a categorical Label. The features are integer counts, byte sizes and microsecond timings, plus floating-point rates, and fall into nine groups: packet and payload size (24), timing and inter-arrival times (15), TCP flags (12), active/idle periods (8), counts (6), bulk statistics (6), rates (4), TCP window (2) and port (1). The target is binary: BENIGN (127,537 raw rows, 44.5%) versus PortScan (158,930, 55.5%). After cleaning, 213,602 flows remain (122,908 BENIGN, 57.5%; 90,694 PortScan, 42.5%). The Monday file (July 3; 529,918 flows, all BENIGN) is used only to test false positives on another day. Splits are in Section 5; the data dictionary is in Appendix A.

Limitations: this release has no timestamp, IP or flow-ID columns, so temporal and host-grouped splits are not possible and the allowlist and slow-scan controls cannot be evaluated on it; traffic comes from one lab and one scanner (Nmap); and scans are far more common than on a real network, so deployed precision would be lower.

3. Data-Quality Assessment

Every check below was run on the raw file before any change (src/01_load_and_audit.py), and each fix is logged with row counts before and after (outputs/tables/cleaning_log.csv). The same cleaning function is applied to the Monday file (529,918 → 502,138 rows).

Table 1. Data-quality checks, evidence and treatment (PortScan file).

Issue checked	Evidence / magnitude	Treatment	Remaining risk
Missing / infinite values	15 NaN and 727 inf cells in Flow Bytes/s and Flow Packets/s (zero-duration flows)	Dropped: 371 rows (245 benign, 126 scan)	Zero-duration flows need explicit handling in deployment
Invalid values	176 rows with negative Flow IAT Min, incl. all 36 with negative Flow Duration (all benign)	Dropped as physically impossible	Low
Sentinel values	Init_Win_bytes = −1 (not observed): 60,210 forward, 76,942 backward rows	Kept; binary missing flags added	Linear models may read −1 as a number
Redundant columns	10 constant; 8 exact copies (e.g. Fwd PSH Flags = SYN Flag Count)	Flagged but kept in the all-features sets; excluded only from the reduced comparison set	Copies split coefficient weight; SYN = Fwd PSH suggests a flag-extraction bug
Duplicates	72,353 rows (25.3%): 68,111 scan (42.9% of scans), 4,242 benign (3.3%)	Removed before splitting	Near-duplicates remain, so random splits stay optimistic
Label quality	0 identical feature vectors with conflicting labels	Check kept in pipeline	Known dataset label errors not detectable (Engelen et al., 2021)

Class imbalance	55.5% scan raw; 42.5% after cleaning	Stratified splits; class weights; recall/FPR reported	Real networks have fewer scans
Outliers	Benign Flow Duration median 31 ms, 99th pct 118.2 s; 22.7% outside IQR fences	Kept (real flows); log transform	Low for tree models
Sampling bias	One lab network, one afternoon, one scanner	Monday outside-day test	High: may not transfer
Leakage / temporal validity	No timestamps; scan share across 20 file-order bins ranges 0.0%–99.4% (one burst)	Dedup before split; selection and scaling fit on train only; time-ordered split; with/without-port test	Random split places flows of one scan on both sides

What this means for modeling. Duplicates matter most: with almost half of scan rows repeated, a model would partly be scored on flows it had already seen, so de-duplication runs before any split. Invalid values affect under 0.2% of rows; −1 window values are kept because an unfinished handshake is informative. Label quality and sampling bias remain limitations, partly tested with the Monday file.

4. Exploratory Data Analysis

EDA used the 62 distinct, non-constant columns (src/03_eda.py). The median scan flow lasts 50 µs with one packet each way and no payload; the median benign flow lasts 31,370 µs (Appendix C, Figure C1). Six features reach a univariate AUC above 0.90 (best: Bwd Packets/s, 0.938), none perfect alone, and 35 pairs correlate above $|r| = 0.95$. A 1,024-byte forward TCP window, a value Nmap commonly uses in SYN probes (Lyon, 2009), marks 49.6% of scan flows and 0.0% of benign flows: a tool fingerprint.

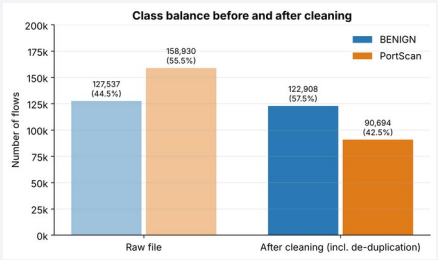


Figure 1. Flow counts by class in the raw PortScan file and after cleaning, with each class's share of the total. *Takeaway: duplicates are concentrated in scan traffic, so cleaning turns benign into the majority class.*

A scanner sends near-identical probes, so 42.9% of scan rows are exact repeats (3.3% of benign). Left in, copies would land in both training and evaluation data and inflate every metric, so de-duplication runs before splitting.

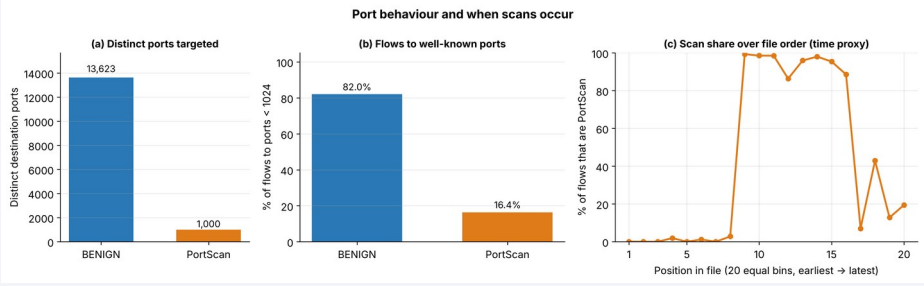


Figure 2. (a) Distinct ports per class, (b) share of flows to ports below 1024, (c) PortScan share across 20 bins of file order (time proxy). *Takeaway: the scanner sweeps a fixed list of ports in one burst.*

Scans touch exactly 1,000 ports almost evenly, consistent with Nmap's default top-1,000 list, while 82.0% of benign flows go to ports below 1024 (mainly 53, 443 and 80) versus 16.4% of scan flows. The port therefore partly identifies the tool, so Destination Port is kept out of the primary feature set and tested separately. Panel (c) shows scans arrive in one block, so a random split places flows of one scan on both sides; hence the time-ordered split. An authorized IT sweep of the same ports would look identical, which is why the known-tester allowlist (Section 7) is needed.

5. Data Preparation and Feature Engineering

Preparation is implemented in src/02_clean.py and src/04_features_and_split.py using scikit-learn (Pedregosa et al., 2011). Every step is deterministic and logged, and anything learned from data (feature ranking and scaling) is fit on the training split only.

Table 2. Preparation steps, feature sets and engineered features.

Step / feature	Implementation	Rationale	Leakage or bias control
Row cleaning	Strip names; drop inf/NaN (371) and negative-time (176) rows	Undefined or impossible values	Rule-based; same on Monday
Sentinel flags (created)	init_win_fwd_missing, init_win_bwd_missing	Dropping −1 rows would lose 21–27% of data	Rule-based
Column handling	All 78 original columns kept; 10 constant and 8 duplicate columns only flagged	Avoid losing any useful feature	No data-driven choice
De-duplication	72,318 exact duplicate rows removed (68,110 scan)	Copies inflate scores	Done before splitting
Feature sets	all_no_port (primary): 77 original columns + 2 flags = 79; all_with_port adds Destination Port and is_well_known_port (81)	Full information; port tested separately	Port excluded from primary set
Reduced comparison set	Drop constant, duplicate and $\|r\| > 0.95$ features (ranked by train AUC): 43 features	Tests whether pruning loses signal	Fit on training split only
Transform and scaling	Signed $\log(1 + \|x\|) \rightarrow$ StandardScaler in a Pipeline (logistic regression)	Heavy tails; −1 sentinel	Scaler fit on train only

Why all columns. Pruning assumes a highly correlated feature adds nothing, but correlation measures only overall linear agreement; two features can still differ on exactly the rare flows that are hard to classify. The team therefore trains on every column. The comparison supports this: several pruned columns receive large weights in the all-features model (Total Backward Packets, rank 7 of 79; Flow IAT Std, rank 10), and validation false alarms fall from 17 to 6 (Appendix C, Table C1). Constant columns receive zero weight, so keeping them costs nothing.

Split strategy. Stratified 70/15/15 (seed 42) after de-duplication: 149,521 train, 32,040 validation and 32,041 test flows, each 42.5% PortScan. A time-ordered split uses file row order (no timestamps); its scan share ranges from 41.6% (train) to 63.7% (validation) and 25.1% (test), confirming the burst, so random splits are optimistic. The time-ordered split and Monday file are the safeguards; host-grouped splits need IP columns. Both test sets stay locked until the Status Report.

6. Baseline Model or Analytical Benchmark

Baselines were trained on the training split and scored on validation (13,604 scan, 18,436 benign flows) and on Monday: (1) majority class; (2) a rule tuned on training data (Flow Duration ≤ 500 µs, Total Fwd Packets ≤ 1, Fwd Packet Length Max ≤ 2 bytes); (3) balanced logistic regression, the proposal's reference baseline, on all features (primary), all plus port, and the reduced set; (4) a depth-4 tree. The primary model reaches precision 0.9996, recall 0.9982, F1 0.9989, PR-AUC 0.9999 and ROC-AUC 0.9998 (plots: outputs/figures, fig6–fig9).

Table 3. Baselines on validation data (Strat. = stratified, Time = time-ordered split) and on 502,138 Monday benign flows.

Model	Split	Precision	Recall	False alarms	FPR	Monday FPR	Meets targets
Majority class	Strat.	0.0000	0.0000	0	0.00%	0.00%	No
Threshold rule	Strat.	0.9541	0.9841	644	3.49%	5.84%	No
Threshold rule	Time	0.9716	0.9848	588	5.05%	5.84%	No
LogReg, all features (primary)	Strat.	0.9996	0.9982	6	0.03%	0.13%	Yes
LogReg, all features (primary)	Time	1.0000	0.9999	1	0.01%	0.03%	Yes
LogReg, all features + port	Strat.	0.9998	0.9987	3	0.02%	0.04%	Yes
LogReg, reduced 43 features	Strat.	0.9987	0.9982	17	0.09%	0.12%	Yes
LogReg, reduced 43 features	Time	1.0000	0.9999	0	0.00%	0.02%	Yes
Decision tree, depth 4, all	Strat.	0.9971	0.9979	40	0.22%	0.73%	Yes

Interpretation. The majority-class model scores 57.5% accuracy yet catches no scans, so accuracy is the wrong headline metric. The rule catches 98.4% of scans but flags 5.05% of benign flows on the time-ordered split and 5.84% on Monday, failing the 5% target on unseen traffic. Logistic regression meets both targets everywhere. Adding the port lowers validation FPR only from 0.03% to 0.02%, so the model is not memorising ports. All features cut validation false alarms from 17 to 6 versus the reduced set at equal recall; on Monday (0.13% vs 0.12%) and the time-ordered split (1 vs 0) the two are effectively tied, so keeping all columns never costs accuracy.

Error analysis. The 24 missed scans mostly targeted ports 80 and 139 and had several forward packets but no reply, so they resemble failed ordinary connections. The 6 false alarms were long-lived flows (median about 24 s) with no reply, on port 443 or port 0. The criteria are reachable with a simple linear model, so the open question is robustness.

7. Challenges and Revised Plan

The proposal and the project review identified the problems below; each now has a concrete solution.

Table 4. Problems identified and their solutions (details in Appendix B).

Problem	Solution	Status
Internal scanners (IT running Nessus or Nmap) look identical to attacks	Known-tester allowlist: approved scanners with source IP, ports, schedule, owner and expiry; alerts suppressed only when all match and kept for audit	Implemented; 6 unit tests pass; needs source-IP data to evaluate
Slow scans spread over hours look normal flow by flow	Per-source features over 5 min, 1 h and 24 h: distinct ports, distinct hosts, share of unanswered connections	Implemented; test flags a 10-minute-interval scan only visible over 24 h
Pruning redundant columns might drop a useful feature	Model on all columns; reduced set kept only as a comparison	Done: validation false alarms 17 → 6
Port number may let the model memorise the scanner's port list	Primary set excludes Destination Port; with-port variant as a check	Done: FPR 0.03% without vs 0.02% with port
2017 data and one scanning tool; scores may reflect tool fingerprints	State the limitation; outside-day test; retrain without TCP-window features	Monday test done; ablation planned
No timestamps or IPs in this release	Obtain CIC's GeneratedLabelledFlows release (adds Timestamp and IPs)	Planned by Oct 25

Table 5. Work planned before the Status Report.

Task	Target date	Lead
Random Forest and XGBoost on all features; 5-fold CV tuning; threshold choice; SHAP	Oct 25 – Nov 1	Ojas
Labelled-flows release: evaluate allowlist and slow-scan features; time/host splits	Oct 25	Riya
Robustness: no TCP-window features; Tuesday–Thursday benign false-positive tests	Nov 1	Riya
Score locked test sets once; write Status Report	Nov 8	Both
Streamlit dashboard: ranked, explained alerts with allowlist status	Nov 22	Riya

References

Canadian Institute for Cybersecurity. (n.d.). Intrusion detection evaluation dataset (CIC-IDS2017). University of New Brunswick. https://www.unb.ca/cic/datasets/ids-2017.html

Engelen, G., Rimmer, V., & Joosen, W. (2021). Troubleshooting an intrusion detection dataset: The CICIDS2017 case study. In 2021 IEEE Security and Privacy Workshops (SPW) (pp. 7–12). IEEE.

Lashkari, A. H., Draper-Gil, G., Mamun, M. S. I., & Ghorbani, A. A. (2017). Characterization of Tor traffic using time based features. In Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP) (pp. 253–262).

Lyon, G. F. (2009). Nmap network scanning: The official Nmap project guide to network discovery and security scanning. Insecure.Com LLC.

NetSentries. (2026). ScanShield [Source code]. GitHub. https://github.com/riyap3111/ScanShield

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

sateeshkumar6289. (n.d.). CICIDS 2017 dataset [Data set]. Kaggle. https://www.kaggle.com/datasets/sateeshkumar6289/cicids-2017-dataset

Scarfone, K., Souppaya, M., Cody, A., & Orebaugh, A. (2008). Technical guide to information security testing and assessment (NIST Special Publication 800-115). National Institute of Standards and Technology.

Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP) (pp. 108–116).

Team Contribution Statement

Team member	Responsibilities and completed work	Evidence / artifact
Riya Sanjiv Pradhan	Data acquisition and audit (01_load_and_audit.py), cleaning pipeline (02_clean.py), data dictionary, EDA and figures (03_eda.py), data-quality section, known-tester allowlist design (allowlist.py, known_scanners.csv); co-wrote the report.	src/01–03, src/allowlist.py, config/, tests/test_allowlist.py, outputs/tables/data_dictionary.csv, cleaning_log.csv, fig1–fig5
Ojas Prashant Suratkar	Feature sets and split design (04_features_and_split.py), baseline models, all-features vs reduced comparison and error analysis (05_baselines.py), slow-scan features (slow_scan.py), results summary and reproducibility script; co-wrote the report.	src/04–06, src/slow_scan.py, tests/test_slow_scan.py, run_all.sh, baseline_results.csv, excluded_feature_check.csv, fig6–fig9

Work was split by pipeline stage, as in the proposal: Riya led the data stages and the alerting side, and Ojas led features and modeling. The decision to keep all columns and the risk solutions in Section 7 were agreed jointly. Each member reviewed the other's scripts and outputs before they were merged, and both reviewed and edited the full report. Participation was even, and no coordination issues needed resolving.

AI-Use Disclosure

Tool	How it was used	What the team verified or changed
Claude (Anthropic)	Assisted with writing and debugging the Python analysis code and with drafting and editing report text.	The team specified the analysis and made all analytical decisions (cleaning rules, keeping all columns, splits, baselines, risk solutions), reviewed the code, re-ran the full pipeline from the raw data, and checked every reported number against the output tables in the repository.

Submission Checklist

Check	Requirement
[x]	Report reflects work completed by the deadline.
[x]	All tables and figures have numbers, captions, and interpretations.
[x]	Data-quality findings include quantitative evidence.
[x]	Baseline results use a clearly identified validation set.
[x]	Repository contains the code or notebooks referenced.
[x]	Contribution statement and AI-use disclosure are complete.

Rubric Self-Check

Criterion	Points	Where the evidence appears
Status, scope, and project management	1	Executive Status Snapshot and Section 1 (pp. 1–2)
Dataset description and data dictionary	2	Section 2 (p. 2); data dictionary in Appendix A (p. 8)
Data-quality assessment	2	Section 3 and Table 1, quantitative checks and treatments (pp. 2–3)
EDA and interpretation	2	Section 4, Figures 1–2 with interpretations (pp. 3–4); Figure C1 (p. 10)
Preparation, features, and baseline	2	Section 5 and Table 2 (p. 4); Section 6 and Table 3 (pp. 4–5); feature check in Appendix C (p. 10)
Challenges, plan, and report quality	1	Section 7, Tables 4–5 (p. 5); risk controls in Appendix B (p. 9); repository link (p. 1); contribution statement (p. 6)

Appendix A. Data Dictionary

Key fields of the PortScan file and engineered variables. The complete 79-field dictionary, with non-null counts, ranges and medians, is in outputs/tables/data_dictionary.csv.

Table A1. Data dictionary (key fields).

Variable / field	Type	Meaning	Role in analysis
Label → is_scan	Categorical → binary	Ground-truth class (BENIGN or PortScan), encoded as 1 = PortScan	Target
Destination Port	Integer (0–65535)	Port the flow was sent to	all_with_port only (leakage check)
Flow Duration	Integer (µs)	Time from first to last packet of the flow	Feature
Total Fwd Packets	Integer	Packets sent by the initiating host	Feature
Total Backward Packets	Integer	Packets sent back by the responding host	Feature (pruned from reduced set; LR rank 7 of 79)
Total Length of Fwd Packets	Integer (bytes)	Payload bytes sent by the initiator	Feature
Fwd Packet Length Max	Integer (bytes)	Largest forward payload in the flow	Feature
Flow Bytes/s, Flow Packets/s	Float	Throughput over the flow; undefined when duration is zero	Feature (inf/NaN rows removed)
Bwd Packets/s	Float	Packets per second from the responder	Feature (best single feature, AUC 0.938)
Flow IAT Mean / Std / Max / Min	Float (µs)	Time between consecutive packets in the flow	Feature
SYN / RST / ACK / PSH Flag Count	Integer	Packets carrying each TCP flag	Feature
Init_Win_bytes_forward / _backward	Integer (bytes); −1 = not observed	Initial TCP window size in each direction	Feature, plus engineered missing flags
Active / Idle Mean, Std, Max, Min	Float (µs)	Time the flow was active before going idle, and idle time	Feature
Bulk-rate fields (6), CWE/URG/Bwd PSH flags (4)	Numeric	Bulk-transfer and rarely used flag statistics; constant in this file	Feature in all-features sets (zero weight)
Subflow fields (4), Avg Fwd Segment Size, Fwd PSH Flags, ECE Flag Count, Fwd Header Length.1	Numeric	Exact copies of other columns	Feature in all-features sets; excluded from reduced set
init_win_fwd_missing / init_win_bwd_missing	Binary (engineered)	1 if the TCP window value is −1	Feature
is_well_known_port	Binary (engineered)	1 if Destination Port < 1024	all_with_port only

Appendix B. Risk Controls: Known-Tester Allowlist and Slow-Scan Features

B.1 Known-tester allowlist. Organizations routinely run authorized scans, such as weekly Nessus vulnerability scans or Nmap inventory sweeps (Scarfone et al., 2008). Without special handling, ScanShield would raise an alert every time one runs, and analysts would learn to ignore the tool. The allowlist is a predefined list of these regular testers, stored in config/known_scanners.csv and applied by src/allowlist.py after the classifier has flagged a flow. Each entry records the fields in Table B1. An alert is suppressed only when the source address, destination port, day, time window and expiry date all match one entry. Suppressed alerts are never deleted: they are kept with status "suppressed" and the entry name, so the security team can audit what the allowlist hid. If an approved host probes a port outside its list or runs outside its window, the alert still fires. This matters because a compromised or spoofed scanner address is a real attack path, and an unconditional IP allowlist would hide it.

Table B1. Allowlist fields (example entries use reserved documentation addresses).

Field	Example	Purpose
name	Weekly Nessus vulnerability scan	Human-readable description of the approved activity
source	192.0.2.10/32	Source IP address or CIDR subnet of the scanner
allowed_ports	1-65535 or 22,80,443	Port range or list the scanner is approved to probe
days, start_time, end_time	Sat, 01:00–05:00	Approved schedule; probes outside it still alert
owner, ticket	IT Security, CHG-1042	Accountable team and change-management reference
expires	2027-06-30	Entry stops matching after this date, forcing periodic review

Six unit tests (tests/test_allowlist.py) confirm that an approved scan inside its window is suppressed, the same host outside its window or on an unapproved port still alerts, unknown sources always alert, subnet entries and expiry dates work, and suppressed alerts stay in the output for audit.

B.2 Slow-scan features. A patient attacker can pace a scan; Nmap's slowest timing templates wait minutes between probes (Lyon, 2009). Each flow then looks ordinary, and a per-flow classifier cannot see the pattern. src/slow_scan.py adds per-source context: for each flow, it counts over the same source's previous 5 minutes, 1 hour and 24 hours how many distinct destination ports and hosts were contacted, and what share of those connections received no reply. A starting rule flags sources with at least 50 distinct ports and at least 80% unanswered connections in 24 hours; the thresholds will be tuned on labelled data. In the unit test (tests/test_slow_scan.py), a synthetic scan of 120 ports at one probe every 10 minutes shows one distinct port in every 5-minute window, but 120 over 24 hours, and is flagged; a normal user making 200 web requests is not. These features will also be added to the classifier as inputs once the GeneratedLabelledFlows release, which has the required source-IP and timestamp columns, is in place.

B.3 Other open issues. Published re-analyses report labeling and flow-construction errors in CICIDS2017 (Engelen et al., 2021); results may be checked on the corrected release if time allows. Because duplicate columns share their weight in logistic regression, coefficients are read in groups rather than one by one, and permutation importance will be used for the tree models.

Appendix C. Supplementary EDA and Feature Check

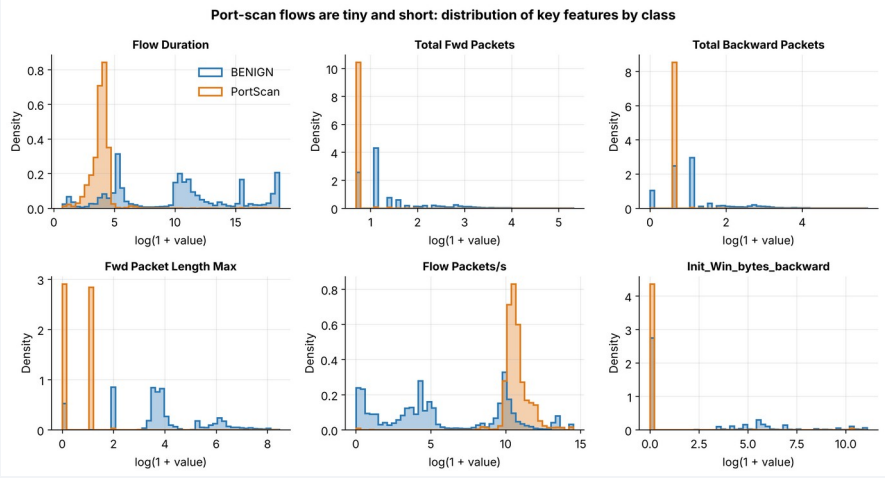


Figure C1. Distributions of six key flow features by class, shown as densities on a log(1 + value) scale. *Takeaway: scan flows are tiny, fast and carry no payload; they overlap with benign traffic only among very short flows.*

Scan flows form narrow spikes (one forward packet, no payload, about 50 µs, very high packet rates), while benign traffic spans several orders of magnitude. The overlap is among short, empty benign flows such as failed or idle connections, which is where false alarms should be expected, and is consistent with the error analysis in Section 6.

Table C1. The 12 highest-weighted columns that the reduced set would have removed (full list: outputs/tables/excluded_feature_check.csv).

Column pruned from the reduced set	Reason pruned	Univariate AUC	Weight rank in all-features LR	Used by tree
Total Backward Packets	|r| > 0.95	0.733	7 of 79	No
Subflow Bwd Packets	copy of Total Backward Packets	0.733	8 of 79	No
Flow IAT Std	|r| > 0.95	0.828	10 of 79	No
Bwd IAT Max	|r| > 0.95	0.792	11 of 79	Yes
Fwd IAT Min	|r| > 0.95	0.864	12 of 79	No
Flow IAT Mean	|r| > 0.95	0.851	13 of 79	No
Bwd IAT Mean	|r| > 0.95	0.792	14 of 79	No
Subflow Fwd Bytes	copy of Total Length of Fwd Packets	0.931	18 of 79	Yes
Fwd IAT Max	|r| > 0.95	0.868	20 of 79	No
Idle Min	|r| > 0.95	0.583	21 of 79	Yes
Total Length of Bwd Packets	|r| > 0.95	0.761	22 of 79	Yes
Subflow Bwd Bytes	copy of Total Length of Bwd Packets	0.761	23 of 79	No

Several pruned columns rank among the 15 strongest weights in the all-features model, and 4 of the 12 shown are used by the depth-4 tree. The ten constant columns receive zero weight. This is the evidence behind the decision to keep all columns.